Boot Controller Architecture & Data Flow

SONIC-MCU

Chunked Burst Transfer, Memory Formatting and Data Flow | SPEC-BOOTFLOW-v7.0

Introduction

The SONIC-MCU utilizes a dedicated hardware state machine to handle the bootstrapping process, circumventing the latency and complexity of a software-based Boot ROM. This Bootloader orchestrates an end-to-end Direct Memory Access (DMA) transfer.

It fetches the firmware image from a passive external SPI Flash chip, coordinates the transaction through an internal APB SPI Master IP, applies necessary endianness corrections, and autonomously writes the formatted instructions and data directly into the system's active I-SRAM and D-SRAM via the AXI-Lite crossbar.

The defining characteristic of the architecture is that the serial fetch and the parallel SRAM write execute as **two independent loops with hardware interlocks in both directions**. The Boot Controller is idle for over 99% of the transfer window, waiting on the SPI shift; every AXI write is absorbed underneath that window and contributes nothing to boot latency.

Firmware Image and Memory Formatting

The firmware integration pipeline is a critical stage. It bridges the gap between how a standard C-compiler outputs memory arrays and how a physical SPI Flash chip expects data to be organized.

Original Word-Addressed Formats (`ebreak_imem.mem` / `ebreak_dmem.mem`)

The compiler initially outputs the compiled C-code into two distinct memory files for instructions and data. These files are formatted strictly as **32-bit, word-addressed hexadecimal files**.

Listing 1: Snippet from `ebreak_imem.mem` (Word-Addressed)

```
@00000000
00000097
@00000001
13408093
```

In this format, the "`@`" symbol denotes the 32-bit word boundary. Because RISC-V is a Little-Endian architecture, the 32-bit instruction `0x00000097` theoretically translates to four consecutive bytes in memory: `0x97`, `0x00`, `0x00`, `0x00`. However, standard SPI Flash arrays operate on a contiguous, 8-bit byte-addressed space, making this native format incompatible with a raw serial flash dump.

The Conversion Pipeline (spi_unified_mem_instr_data.py)

To resolve this, the build flow invokes a Python conversion script. This script executes three vital formatting transformations:

1. **Address Translation:** It maps word addresses to byte addresses (e.g., Word Address 0x00000001 logically maps to Byte Address 0x00000004). It aggressively pads any skipped addresses in the input file with 00 to guarantee a 1:1 physical layout.
2. **Little-Endian Byte Unpacking:** It intercepts the 32-bit hex strings and decomposes them into Little-Endian byte arrays. The word 13408093 is sliced and rearranged as 93, 80, 40, 13.
3. **Unified Concatenation:** It merges the converted Instruction memory and Data memory into a single, continuous block, eliminating the need for complex multi-file handling during flash programming.

Note

The contiguity guaranteed by step 3 is a hard prerequisite for the burst transfer architecture. Because the image occupies an unbroken byte range in flash, a single Read Data command streams across the entire ISRAM/DSRAM boundary without interruption; the Boot Controller retargets the AXI destination address mid-burst without touching the SPI transaction.

The Combined Output (ebreak_combined.mem)

The result of the script is the final flash image file. It lacks "@" address tags because it operates implicitly: line N corresponds exactly to byte address N in the SPI flash memory space.

Listing 2: Snippet from ebreak_combined.mem (Byte-Addressed, Little Endian)

```
97
00
00
00
93
80
40
13
```

One-Time Programmable Formatting (s25fs512sOTP.mem)

Conversely, internal hardware simulation models (like the S25fs512s simulation model) utilize specialized memory maps for their One-Time Programmable (OTP) security registers. The s25fs512sOTP.mem file is structured distinctly to interface with the simulator's internal loading routines:

Listing 3: Snippet from s25fs512sOTP.mem

```
@000
00
01
02
@010
9F
```

While this file is also byte-oriented, the simulator's $readmemh parser expects explicit byte-address anchors (e.g., @000, @010) to map these isolated security bytes directly into internal status registers rather than the main sequential flash array.

Important

Necessity of the OTP File: The OTP memory contains critical configuration bytes, non-volatile lock bits, and device identification parameters. In physical hardware, these are programmed at the factory or during device provisioning. In the verification environment, the `s25fs512s` simulation model strictly requires the `s25fs512sOTP.mem` file to preload these internal states. If this file is omitted or incorrectly formatted, the flash model will initialize in an undefined or locked state, causing it to reject incoming SPI read commands or assert false protection faults, leading to simulated boot failures.

Chunked Burst Transfer Architecture

Protocol Overhead and the Case for Bursts

Every SPI Flash read transaction carries a fixed protocol header before any payload is delivered: an 8-bit Read opcode followed by a 24-bit flash address. If a transaction were to deliver only a single 32-bit firmware word, that header would cost as much as the payload itself:

$$\underbrace{8}_{\text{CMD}} + \underbrace{24}_{\text{ADDR}} + \underbrace{32}_{\text{DATA}} = 64 \text{ SPI bits to move 32 bits of firmware}$$

Half of all serial traffic would be protocol overhead, repeated once per word, alongside two additional APB register writes per word to reprogram the flash address and re-trigger the transfer. Since boot latency on this device is dominated almost entirely by serial shift time, eliminating that header is the single largest lever available.

The Mechanism: Flash Address Auto-Increment

SPI NOR opcode `0x03` (Read Data) auto-increments its internal address pointer for as long as chip-select remains asserted. A single command+address therefore streams an arbitrarily long run of consecutive bytes, and the Boot Controller issues one header per *chunk* of many words rather than per word.

Three properties of the `apb_spi_master` IP make this directly exploitable without modification to the SPI IP:

1. **The RX path packetises automatically.** `spi_master_rx` raises `data_valid` every 32 shifted bits (`counter[4:0] == 5'b11111`), pushing a completed word into the RX FIFO. A long data phase therefore produces a stream of words, not one oversized register.
2. **The RX path self-throttles.** If the RX FIFO has no space, `spi_master_rx` drops `clk_en_o`, gating SCK mid-burst, and resumes when space frees. Overflow is structurally impossible at any drain rate.
3. **The APB path stalls the consumer.** An RX FIFO read issued while the FIFO is empty holds `PREADY` low, parking the Boot FSM until data arrives.

Burst Length Ceiling

The burst length is bounded by the SPI master's length register, not by the flash device:

Table 1: Constraints on maximum burst length

Constraint	Source
`spi_data_len` is 16 bits	`spi_master_apb_if.sv` (register field, written from `PWDATA[31:16]`)
`counter_rx` is `[15:0]`	`spi_master_controller.sv`, loaded directly from `spi_data_len`
Maximum burst	$65{,}535$ bits $\div$ 32 bits/word $=$ **2,047 words per transaction**
SoC memory capacity	ISRAM $+$ DSRAM $= 8192 + 8192 =$ **16,384 words**

Because the SoC's memory capacity exceeds a single transaction by $8\times$, the transfer is split into chunks of `MAX_CHUNK_WORDS`, with the final chunk shortened to whatever remains.

Important

Guarded failure mode. Because the length field is 16 bits wide, a programmed length of exactly 2,048 words ($65{,}536$ bits) truncates to **zero**. `spi_master_controller` gates its data phase on `spi_data_len != 0`, so a zero-length program skips the data phase entirely — producing a boot that fetches nothing, with no error and no timeout. An elaboration-time assertion in `bootloader.sv` rejects any `MAX_CHUNK_WORDS` outside the range $1 \ldots 2047$, making this condition impossible to build.

Parametric Scaling

Every counter width is derived from the module parameters, so the design tracks any SRAM depth without RTL edits:

Table 2: Chunk count across SRAM configurations (`MAX_CHUNK_WORDS` $= 1024$)

SRAM depth	Max image (words)	Chunks	RTL change required
2 K $+$ 2 K	1,024	1	None
4 K $+$ 4 K	2,048	2	None
8 K $+$ 8 K	4,096	4	None
16 K $+$ 16 K	8,192	8	None
32 K $+$ 32 K	**16,384**	**16**	**None**

System Reset and Boot Orchestration

In the SONIC-MCU architecture, the Bootloader does not act merely as a data pump; it serves as the Master Reset and Boot Controller for the entire System-on-Chip (SoC).

Waking up the RISC-V core and the Bootloader simultaneously without sequencing would induce immediate contention on the AXI-Lite crossbar. To prevent deadlocks and mitigate dynamic power glitches from uninitialized logic matrices, the Bootloader hardware actively halts the core and non-essential peripherals during memory population.

Dual Reset Topology and Memory Retention

The system provides two distinct external reset paths that dictate the Bootloader's execution flow:

- **Cold Boot (`hard_rst_ni`):** Initiated during a full hardware power-up. The Bootloader executes a complete initialization: sanitizing the SRAM, waiting for external flash power-up, configuring the SPI Master via APB, and orchestrating the chunked DMA fetch loop to populate memory.
- **Warm Boot (`soft_rst_ni`):** Initiated for a rapid system restart. This path protects existing SRAM integrity by clamping the SRAM write-enable signals to zero. The FSM skips the SPI flash fetch entirely, allowing the CPU to instantly re-execute previously loaded firmware. Measured cost: **6 cycles**.

Internal Reset Outputs

Three internally generated resets are driven by the Boot FSM. `spi_rst_ni` is released *early* — well before the SPI IP is first accessed — so that the master observes a genuine falling-to-rising transition rather than an undefined-to-high transition, which would leave its internal negedge-triggered registers uninitialized:

Table 3: Internally generated reset outputs

Signal	Drives	Release point
`spi_rst_ni_o`	`apb_spi_master`	On leaving `S_RESET` (early)
`core_rst_ni_o`	RISC-V CPU	`S_CLEAN_RELEASE`
`periph_rst_ni_o`	I2C, Timer, UART, GPIO	`S_CLEAN_RELEASE`

Boot Controller State Machine

The flowchart below shows the **two nested loops** that structure the transfer: an outer per-chunk loop that programs and triggers a burst, and an inner per-word loop that drains it.

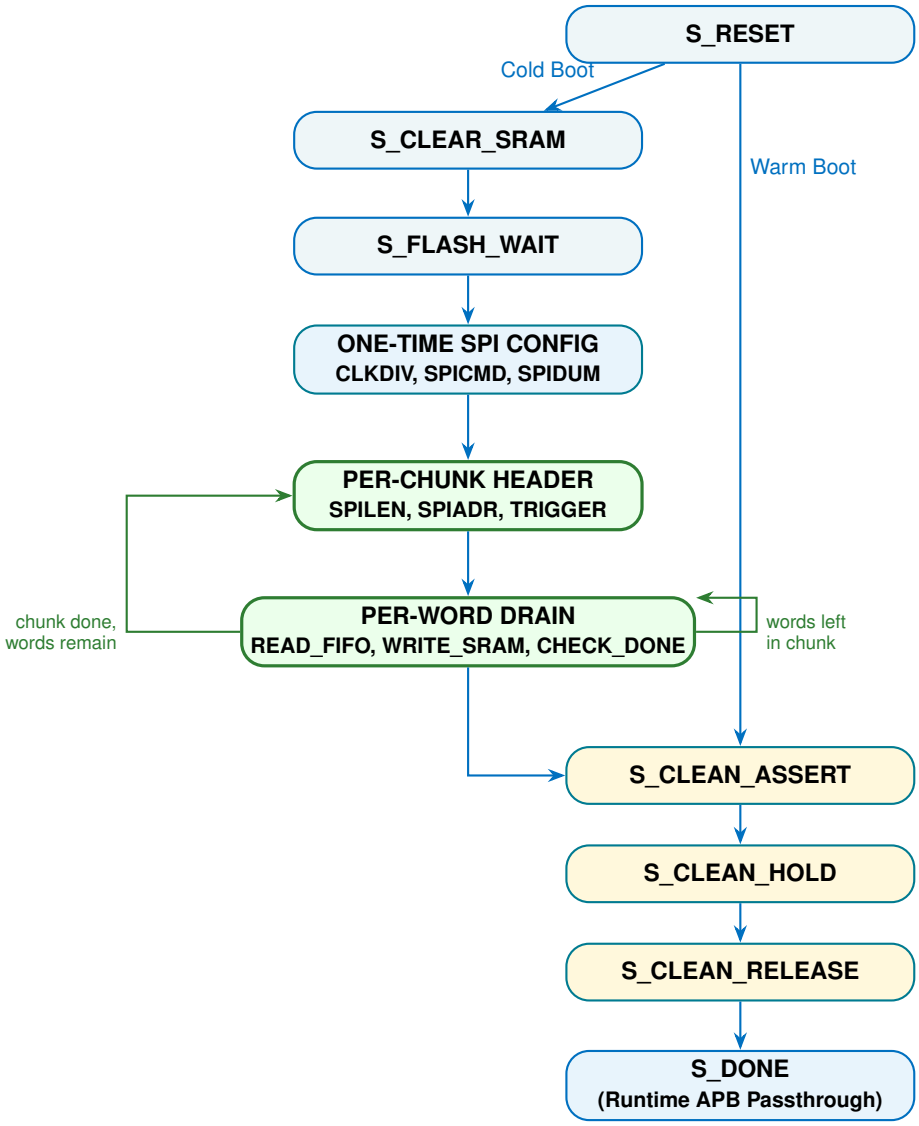


Figure 1: Boot and Reset Sequence FSM. Green blocks are the nested chunk and word loops.

Full State Enumeration

Table 4: Boot FSM states and execution frequency

#	State	Runs	Function
0	S_RESET	once	Branch cold vs. warm on warm_boot_q
1	S_CLEAR_SRAM	once	AXI write of zero to every ISRAM then DSRAM word
2	S_FLASH_WAIT	once	Flash power-on recovery countdown
3	S_INIT_CLKDIV	once	APB: SPI clock divider
4	S_INIT_SPICMD	once	APB: Read opcode 0x03, MSB-aligned
6	S_INIT_SPIDUM	once	APB: dummy cycles = 0
5	S_INIT_SPILEN	**per chunk**	APB: burst geometry, data field = chunk_size×32
7	S_CFG_SPIADR	**per chunk**	APB: flash byte address of this burst
8	S_TRIGGER	**per chunk**	APB: assert CS, start SCK
9	S_READ_FIFO	**per word**	APB read of RX FIFO; sleeps on PREADY=0
10	S_WRITE_SRAM	**per word**	AXI write, waits for b_valid
11	S_CHECK_DONE	**per word**	Advance counters; three-way branch
12	S_CLEAN_ASSERT	once	All three resets LOW
13	S_CLEAN_HOLD	once	Hold for RESET_HOLD_CYCLES
14	S_CLEAN_RELEASE	once	All three resets HIGH, same edge
15	S_DONE	terminal	boot_done_o=1, APB passthrough

Note

S_INIT_SPILEN is listed out of numeric order deliberately. Although it holds enumeration value 5, it is not part of the one-time configuration chain: because the final chunk is shorter than the others, the burst length must be reprogrammed at the head of *every* chunk. The traversal order is CLKDIV → SPICMD → SPIDUM → SPILEN → SPIADR → TRIGGER.

Post-Load Clean-Slate Sequence

A critical requirement of the boot architecture is ensuring that no leftover bootloader configuration — such as the SPI Clock Divider or an active Chip Select state — leaks into the CPU's runtime execution environment.

Before entering the terminal S_DONE state in either a cold or warm boot, the FSM traverses the clean-slate sequence (S_CLEAN_ASSERT → S_CLEAN_HOLD → S_CLEAN_RELEASE). This sequence simultaneously pulses core_rst_ni, periph_rst_ni, and spi_rst_ni LOW for RESET_HOLD_CYCLES clock cycles, then releases all three on the same edge.

Listing 4: Reset output management, keyed on the next-state decode

```
unique case (state_d)
  S_CLEAR_SRAM, S_FLASH_WAIT, S_INIT_CLKDIV: begin
    spi_rst_q <= 1'b1;              // early release: real negedge->posedge
  end
  S_CLEAN_ASSERT, S_CLEAN_HOLD: begin
    core_rst_q   <= 1'b0;
    periph_rst_q <= 1'b0;
    spi_rst_q    <= 1'b0;
  end
  S_CLEAN_RELEASE, S_DONE: begin
    core_rst_q   <= 1'b1;
    periph_rst_q <= 1'b1;
```

```
    spi_rst_q     <= 1'b1;
  end
  default: ;
endcase
```

Here `state_d` is the combinational next state, so the reset outputs change on the same edge the FSM advances. `core_rst_q`, `periph_rst_q` and `spi_rst_q` are the registered forms of the three reset outputs described above; all are active-LOW.

Dynamic APB Multiplexing

The Bootloader physically sits between the system's APB Bridge (Slot 4) and the `apb_spi_master` IP.

- **During Boot (`boot_active = 1`):** The FSM directly drives the APB master port to configure the SPI registers and execute the hardware DMA. Any incoming APB traffic from the slave port is safely met with an idle response.
- **At Runtime (`S_DONE`):** The FSM relinquishes bus control, creating a transparent passthrough. The CPU communicates directly with the SPI Master over the AXI-Lite crossbar and APB bridge as if the Bootloader hardware were not present.

Decoupled Hardware Synchronization

The boot flow relies on two Finite State Machines executing simultaneously, synchronized entirely in hardware with no software polling.

Bidirectional Backpressure

Table 5: Hardware interlocks between the two loops

Condition	Mechanism	Effect
RX FIFO empty	`PREADY` held LOW on the FIFO read	Boot FSM sleeps in `S_READ_FIFO`. No polling loop, no timer.
RX FIFO full	`spi_master_rx` drops `clk_en_o`	**SCK is gated mid-burst**; the shift resumes when the consumer drains. Overflow is structurally impossible.

Note

The second interlock is what makes an arbitrarily long burst safe. The measured drain margin is roughly $100\times$: the SPI master produces one word every 320 cycles, while the Boot FSM consumes one in 3 cycles once data is present. The clock-gating path therefore never engages in practice, but it bounds the design correctly regardless of clock ratio, flash speed, or crossbar latency.

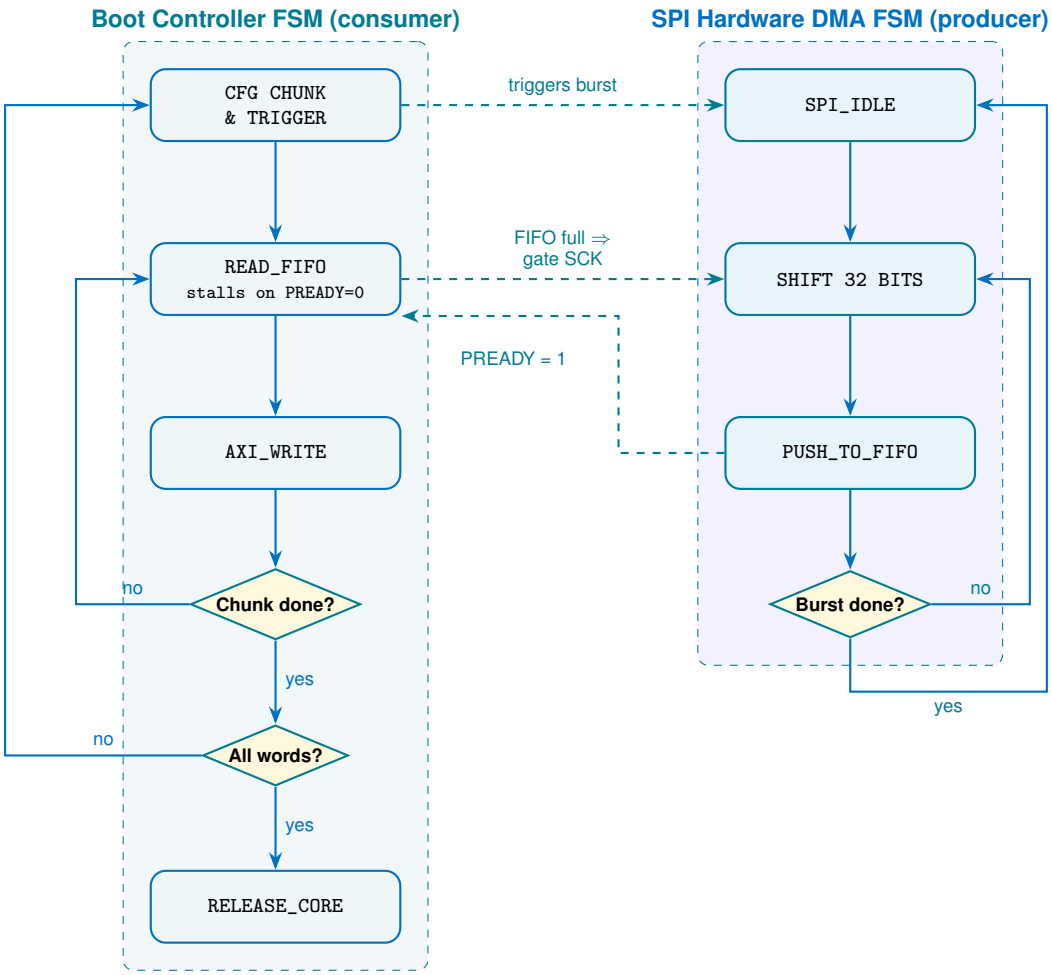


Figure 2: Parallel FSMs with bidirectional hardware backpressure

End-to-End Data Flow Pipeline

The journey of a single 32-bit firmware word follows this route from non-volatile storage to active system memory. Stages 1–3 for word N execute concurrently with the serial shift of word $N+1$:

1. **Stage 1: Flash to SPI Master (Physical Layer):** The external Flash chip shifts data out serially on its `SO` pin. This enters the SoC via the SPI Master's `spi_sdi1_w` pin. The Master captures 32 bits and pushes them to its `RX_FIFO`, *without releasing chip-select* — the next word begins shifting immediately.

2. **Stage 2: SPI Master to Bootloader (APB Protocol):** The Bootloader, stalled in `S_READ_FIFO`, sees `PREADY` go HIGH. It captures the data from the FIFO across the APB bus.

3. **Stage 3: Endianness Correction & AXI-Lite Target:** Because the data was packed byte-by-byte into the flash (Byte 0 first), it must be reversed. The Bootloader swaps the byte lanes, reconstructing the native Little-Endian format, and drives it onto the AXI-Lite `WDATA` channel with the target SRAM address on `AWADDR`.

4. **Stage 4: Destination Retargeting:** On the word where `word_cnt_q` reaches `NUM_INSTR_WORDS`, the destination pointer jumps from the ISRAM base to the DSRAM base. **The SPI burst is unaffected** and continues streaming, because the flash image is contiguous across that boundary.

Hardware Synchronization Timing (Waveform)

The waveform shows a burst in steady state. Chip-select stays asserted across many words; the RX FIFO fills and drains repeatedly inside one transaction, and the AXI write for each word completes while the next word is still shifting.

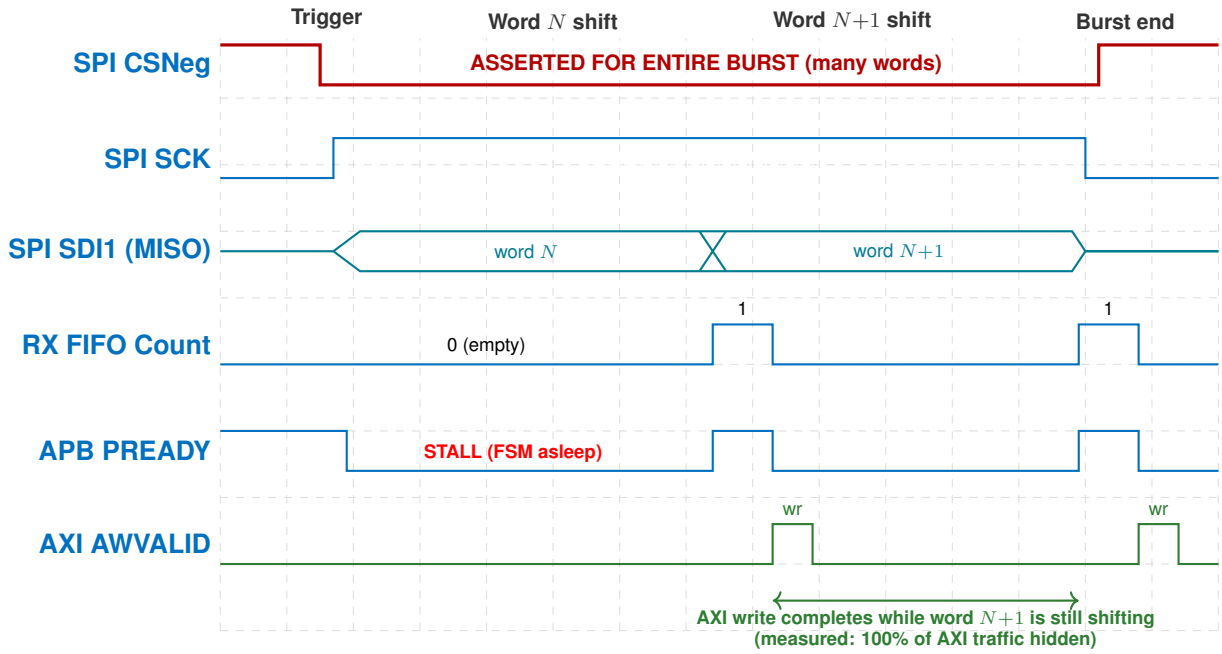


Figure 3: Steady-state burst timing. Chip-select remains asserted; the AXI write for word N overlaps the shift of word $N+1$.

RTL Implementation & Code Flow

The bootloader is implemented in SystemVerilog using a multi-process Finite State Machine architecture, separated into sequential state updates, combinational next-state decoding, and active bus driving blocks.

Signal and Parameter Reference

The listings in this section use the following identifiers. Signals suffixed `_q` are registered; `_d` denotes the combinational next value.

Table 6: Identifiers used in the RTL listings

Identifier	Kind	Meaning
`TOTAL_WORDS`	parameter	Firmware size in 32-bit words, `NUM_INSTR_WORDS` + `NUM_DATA_WORDS`
`NUM_INSTR_WORDS`	parameter	Instruction words destined for ISRAM
`MAX_CHUNK_WORDS`	parameter	Words fetched per SPI burst
`DATA_LEN_BITS`	parameter	Bits per firmware word (32)
`CMD_LEN_BITS`	parameter	Opcode width in bits (8)
`ADDR_LEN_BITS`	parameter	Flash address width in bits (24)
`DSRAM_BASE_ADDR`	parameter	Base address of the data SRAM region
`WORD_CNT_W`	localparam	Bit-width of the global word counter
`CHUNK_CNT_W`	localparam	Bit-width of the per-chunk counter
`REM_W`	localparam	Comparison width, the wider of the two above
`CHUNK_WORDS_LIMIT`	localparam	Largest burst the length register can express (2,047)
`state_q / state_d`	FSM	Current and next FSM state
`warm_boot_q`	flag	Set by `soft_rst_ni`; selects the warm-boot path
`word_cnt_q`	counter	Words transferred so far, across the whole image
`chunk_left_q`	counter	Words still expected from the in-flight burst
`flash_addr_q`	pointer	Next flash byte address to read
`dest_addr_q`	pointer	Next SRAM byte address to write
`rxfifo_data_q`	data	Byte-swapped word captured from the RX FIFO
`words_remaining`	wire	`TOTAL_WORDS` − `word_cnt_q`
`chunk_size`	wire	Words in the burst about to be programmed
`chunk_len_bits`	wire	`chunk_size` × `DATA_LEN_BITS`, written to SPILEN
`apb_phase_q`	handshake	APB phase tracker: 0 = setup, 1 = access
`apb_mst_resp_i.pready`	handshake	APB slave ready; LOW while the RX FIFO is empty
`axi_resp_i.b_valid`	handshake	AXI-Lite write response valid: the SRAM write landed
`apb_boot_paddr / _pwdata`	bus	Address and data driven onto the APB master port

Derived Chunk Constants

Listing 5: Derived chunk constants

```
localparam int unsigned SPI_LEN_MAX_BITS  = 65535;
localparam int unsigned CHUNK_WORDS_LIMIT = SPI_LEN_MAX_BITS / int'(DATA_LEN_BITS);
localparam int unsigned CHUNK_CNT_W       = $clog2(MAX_CHUNK_WORDS + 1);

// Comparison width must cover BOTH the global word counter and the chunk
// counter. A firmware image smaller than MAX_CHUNK_WORDS makes WORD_CNT_W the
```

```
// narrower of the two (e.g. TOTAL_WORDS=64 -> 7 bits vs CHUNK_CNT_W=11), so
// sizing on either one alone yields an out-of-range part-select.
localparam int unsigned REM_W = ((WORD_CNT_W + 1) > CHUNK_CNT_W)
                              ? (WORD_CNT_W + 1) : CHUNK_CNT_W;
```

Chunk Sizing

Each burst is either a full `MAX_CHUNK_WORDS` transfer or, on the final chunk, whatever remains of the image. `chunk_len_bits` is the value ultimately programmed into the SPI master's length register.

Listing 6: Chunk sizing (combinational)

```
always_comb begin
  words_remaining = REM_W'(TOTAL_WORDS) - REM_W'(word_cnt_q);
  chunk_size_wide = (words_remaining >= REM_W'(MAX_CHUNK_WORDS))
                  ? REM_W'(MAX_CHUNK_WORDS)
                  : words_remaining;
  chunk_size      = chunk_size_wide[CHUNK_CNT_W-1:0];
  chunk_len_bits  = 16'(chunk_size) * DATA_LEN_BITS;
end
```

State Register and Reset Handling

The sequential block is clocked on the positive edge of the system clock and handles asynchronous assertion of either reset pin. Both paths enter `S_RESET`; the `warm_boot_q` flag is what distinguishes them, and the next-state logic uses it to select the short path.

Listing 7: FSM state register update

```
always_ff @(posedge clk_i or negedge hard_rst_ni or negedge soft_rst_ni) begin
  if (!hard_rst_ni) begin
    state_q      <= S_RESET;   // Cold boot: start from scratch
    warm_boot_q  <= 1'b0;
    chunk_left_q <= '0;
    // ... reset all counters, hold core/periph/spi resets LOW
  end else if (!soft_rst_ni) begin
    state_q      <= S_RESET;   // Warm boot: enter S_RESET with the flag set;
    warm_boot_q  <= 1'b1;      // next-state logic jumps to S_CLEAN_ASSERT
    chunk_left_q <= '0;
  end else begin
    state_q      <= state_d;
  end
end
```

Next State Logic (Combinational FSM)

The transition logic advances only on acknowledged bus handshakes. An APB transaction completes when `apb_phase_q` indicates the access phase *and* `pready` is asserted — the phase term prevents the FSM advancing during the APB setup phase. An AXI write completes on `b_valid`, the write *response*, so the FSM cannot proceed before the SRAM write has actually landed.

Listing 8: Combinational next-state evaluation

```
// --- One-time SPI config. SPILEN is NOT here: it carries the per-chunk
//     burst length and is reprogrammed at the head of every chunk.
S_INIT_CLKDIV: if (apb_phase_q && apb_mst_resp_i.pready) state_d = S_INIT_SPICMD;
S_INIT_SPICMD: if (apb_phase_q && apb_mst_resp_i.pready) state_d = S_INIT_SPIDUM;
S_INIT_SPIDUM: if (apb_phase_q && apb_mst_resp_i.pready) state_d = S_INIT_SPILEN;

// --- Per-CHUNK burst setup (3 APB writes per chunk, not per word)
S_INIT_SPILEN: if (apb_phase_q && apb_mst_resp_i.pready) state_d = S_CFG_SPIADR;
S_CFG_SPIADR:  if (apb_phase_q && apb_mst_resp_i.pready) state_d = S_TRIGGER;
S_TRIGGER:     if (apb_phase_q && apb_mst_resp_i.pready) state_d = S_READ_FIFO;

// --- Per-WORD drain loop (runs chunk_size times per burst)
```

```
S_READ_FIFO:   if (apb_phase_q && apb_mst_resp_i.pready) state_d = S_WRITE_SRAM;
S_WRITE_SRAM:  if (axi_resp_i.b_valid)                   state_d = S_CHECK_DONE;

S_CHECK_DONE: begin
  if (word_cnt_q + 1 >= TOTAL_WORDS[WORD_CNT_W-1:0])
    state_d = S_CLEAN_ASSERT;     // whole image transferred
  else if (chunk_left_q == CHUNK_CNT_W'(1))
    state_d = S_INIT_SPILEN;      // burst exhausted -> set up next chunk
  else
    state_d = S_READ_FIFO;        // stay inside the current burst
end
```

Note

The ordering of the three tests inside `S_CHECK_DONE` is load-bearing. The total-words test must precede the chunk test: for an image whose length is an exact multiple of `MAX_CHUNK_WORDS` both conditions become true on the same word, and evaluating the chunk test first would program a final zero-length burst.

Per-Chunk Burst Length Programming

`S_INIT_SPILEN` packs the transaction geometry into a single APB write. The command and address widths are constant for the whole boot; only the data field varies, carrying the length of the burst about to be launched.

Listing 9: SPILEN carries the burst length

```
S_INIT_SPILEN: begin
  apb_boot_paddr[11:0] = REG_SPILEN_OFF;
  apb_boot_pwdata      = {chunk_len_bits, 2'b00, ADDR_LEN_BITS,
                          2'b00, CMD_LEN_BITS};
end
```

Counter and Pointer Management

Listing 10: Chunk and address counter management

```
if (state_q == S_CHECK_DONE) begin
  // flash_addr_q advances per word even though the flash auto-increments
  // within a burst: this keeps it pointing at the correct byte address to
  // program into SPIADR at the head of the NEXT chunk.
  flash_addr_q <= flash_addr_q + 24'd4;
  word_cnt_q   <= word_cnt_q + 1'b1;
  if (chunk_left_q != '0)
    chunk_left_q <= chunk_left_q - 1'b1;
  if (word_cnt_q + 1 == NUM_INSTR_WORDS[WORD_CNT_W-1:0])
    dest_addr_q <= DSRAM_BASE_ADDR;      // ISRAM -> DSRAM crossover
  else
    dest_addr_q <= dest_addr_q + AXI_ADDR_WIDTH'(4);
end

// Armed when the burst length is programmed
if (state_q == S_INIT_SPILEN && apb_phase_q && apb_mst_resp_i.pready)
  chunk_left_q <= chunk_size;
```

`chunk_left_q` is loaded with `chunk_size` the moment the length register write is acknowledged, then decremented once per drained word. It reaching 1 in `S_CHECK_DONE` is the signal that the current burst is exhausted. Because the FSM leaves the drain loop at that point, the counter can never be over-decremented.

Endianness Correction and AXI Routing

The flash stores each firmware word least-significant-byte first, while the SPI master shifts every byte MSB-first. The word arriving in the RX FIFO is therefore byte-reversed relative to the RISC-V representation. On

the cycle PREADY asserts during S_READ_FIFO, the Boot Controller reverses the four byte lanes of prdata — lane [7:0] to [31:24], [15:8] to [23:16], and so on — and latches the result into rxfifo_data_q. That register drives the AXI-Lite w.data channel directly, so the word placed in SRAM is already in native Little-Endian form.

Listing 11: RX FIFO capture with byte-lane reversal

```
// Capture and byte-swap on the cycle the APB read completes.
//   apb_phase_q  : 1 during the APB access phase (0 = setup)
//   prdata       : raw RX FIFO word, MSB-first off the wire
//   rxfifo_data_q: byte-reversed result, held for the AXI write
if (state_q == S_READ_FIFO && apb_phase_q && apb_mst_resp_i.pready)
  rxfifo_data_q <= {apb_mst_resp_i.prdata[ 7: 0],
                    apb_mst_resp_i.prdata[15: 8],
                    apb_mst_resp_i.prdata[23:16],
                    apb_mst_resp_i.prdata[31:24]};
```

The AXI-Lite request is assembled combinationally from that register. The write-valid strobes are qualified by per-channel completion latches (aw_done_q, w_done_q), so each channel's valid deasserts as soon as *that* channel's handshake completes, even if the other is still outstanding. Both latches clear together on b_valid, arming the pair for the next word.

Listing 12: AXI-Lite write request assembly

```
// axi_wr_addr / axi_wr_data are selected by state:
//   S_CLEAR_SRAM -> clear_addr_q, 32'h0
//   S_WRITE_SRAM -> dest_addr_q,  rxfifo_data_q
always_comb begin
  axi_req_o           = '0;
  axi_req_o.aw.addr   = axi_wr_addr;
  axi_req_o.aw.prot   = axi_pkg::prot_t'(3'b001);   // privileged access
  axi_req_o.aw_valid  = is_axi_write_state(state_q) && !aw_done_q;
  axi_req_o.w.data    = axi_wr_data;
  axi_req_o.w.strb    = {(AXI_DATA_WIDTH/8){1'b1}}; // full 32-bit word
  axi_req_o.w_valid   = is_axi_write_state(state_q) && !w_done_q;
  axi_req_o.b_ready   = is_axi_write_state(state_q);
end

// Per-channel completion tracking; both clear on the write response.
if (is_axi_write_state(state_q)) begin
  if (axi_req_o.aw_valid && axi_resp_i.aw_ready) aw_done_q <= 1'b1;
  if (axi_req_o.w_valid  && axi_resp_i.w_ready)  w_done_q  <= 1'b1;
  if (axi_resp_i.b_valid) begin
    aw_done_q <= 1'b0;
    w_done_q  <= 1'b0;
  end
end
```

Elaboration-Time Guards

Two assertions in bootloader.sv constrain the parameter space at elaboration rather than allowing an invalid configuration to be built.

The first requires DATA_LEN_BITS to equal 32. This is not a free parameter: the SPI master assembles its RX stream into fixed 32-bit entries, so any other value would desynchronise the FIFO packetisation from the word size the Boot Controller expects.

The second requires MAX_CHUNK_WORDS to lie between 1 and CHUNK_WORDS_LIMIT (2,047 at 32 bits per word). Without it, a larger burst length would silently truncate modulo $65{,}536$ when written to the SPI length register. At exactly 2,048 words the register would receive zero, and because the SPI controller gates its data phase on a non-zero length, the transfer would be skipped entirely — yielding a boot that fetches nothing, reports no error, and never times out.

Listing 13: Parameter assertions

```
// CHUNK_WORDS_LIMIT = 65535 / DATA_LEN_BITS = 2047 words at 32 bits/word,
// set by the 16-bit width of the SPI master's length register.
initial begin
  assert (DATA_LEN_BITS == 16'd32) else
    $error("bootloader: DATA_LEN_BITS must be 32 - the SPI master assembles the RX stream
    into 32-bit entries (got %0d)",
```

```
            DATA_LEN_BITS);

  assert (MAX_CHUNK_WORDS >= 1 && MAX_CHUNK_WORDS <= CHUNK_WORDS_LIMIT) else
    $error("bootloader: MAX_CHUNK_WORDS (%0d) must be in 1..%0d for DATA_LEN_BITS=%0d",
           MAX_CHUNK_WORDS, CHUNK_WORDS_LIMIT, DATA_LEN_BITS);
end
```

SPI Clock Generation

The SPI serial clock is derived from the system clock inside `spi_master_clkgen`. Its internal counter runs to the programmed divider target and then toggles the output, producing a half-period of $(\texttt{SPI_CLK_DIV} + 1)$ system clocks:

$$T_{SCK} = 2 \times (\texttt{SPI_CLK_DIV} + 1) \times T_{clk}$$

At the benchmark settings of $T_{clk} = 10$ ns and $\texttt{SPI_CLK_DIV} = 4$, this gives $T_{SCK} = 100$ ns, a serial clock of **10 MHz**, and a transfer rate of **10 system cycles per SPI bit**. Measured chip-select assertion time during the benchmark confirms this rate to within one cycle.

Boot Performance & Hardware Metrics

Measurements below were captured by `bootloader_tb_all_testcases_32KB_sram.sv` on Xcelium 22.09, all testcases passing.

Configuration Under Test

Table 7: Benchmark configuration

Parameter	Value	Note
System clock T_{clk}	10 ns	100 MHz
`SPI_CLK_DIV`	4	$\Rightarrow$ 10 MHz SCK
`ISRAM_NUM_WORDS`	8,192	32 KB
`DSRAM_NUM_WORDS`	8,192	32 KB
Clear-pass words	16,384	ISRAM + DSRAM
`NUM_INSTR_WORDS`	375	
`NUM_DATA_WORDS`	1,164	
Firmware image	1,539 words	6,156 bytes
`MAX_CHUNK_WORDS`	1,024	$\Rightarrow$ 2 bursts (1024 + 515)
`FLASH_POWERUP_CYCLES`	30,000	S25FS512S $t_{PU} = 300\,\mu$s

Boot Phase Breakdown

Table 8: Cycle-accurate boot latency breakdown

Boot Phase	Cycles	Time	% Boot
S_RESET entry	6	0.06 μs	0.00%
SRAM clear (16,384 words)	32,768	327.68 μs	5.89%
Flash power-up (t_{PU})	30,000	300.00 μs	5.40%
SPI config (one-time)	6	0.06 μs	0.00%
Burst setup (2 chunks)	12	0.12 μs	0.00%
Fetch drain (1,539 words)	**493,118**	**4,931.18 μs**	**88.70%**
Clean-slate reset	6	0.06 μs	0.00%
TOTAL BOOT LATENCY	**555,917**	**5.56 ms**	**100%**

Per-State Cycle Accounting

Table 9: Cycles spent in each FSM state during one cold boot

State	Cycles	%	Per-instance
S_RESET	6	0.00%	—
S_CLEAR_SRAM	32,768	5.89%	2.0 / word
S_FLASH_WAIT	30,000	5.40%	—
S_INIT_CLKDIV	2	0.00%	2
S_INIT_SPICMD	2	0.00%	2
S_INIT_SPIDUM	2	0.00%	2
S_INIT_SPILEN	4	0.00%	2 / chunk
S_CFG_SPIADR	4	0.00%	2 / chunk
S_TRIGGER	4	0.00%	2 / chunk
S_READ_FIFO	**488,501**	**87.87%**	**317.4 / word**
S_WRITE_SRAM	3,078	0.55%	2.0 / word
S_CHECK_DONE	1,539	0.28%	1.0 / word
S_CLEAN_ASSERT	1	0.00%	—
S_CLEAN_HOLD	4	0.00%	—
S_CLEAN_RELEASE	1	0.00%	—

Note

S_READ_FIFO accounts for 87.87% of total boot. This is not Boot Controller work — it is the FSM asleep on PREADY, waiting for the SPI shift. Total FSM configuration and control overhead across the entire boot is **30 cycles**. The design is completely SPI-bound.

SPI / AXI Concurrency

Table 10: Measured overlap between the serial fetch and the parallel write

Quantity	Cycles	Share
SPI burst in flight (CSNeg asserted)	493,119	88.70% of boot
AXI write activity (`S_WRITE_SRAM`)	3,078	0.55% of boot
— overlapped with an active burst	3,078	**100.00% of AXI**
— **not** overlapped (added to critical path)	**0**	**0.00%**
Effective cost per firmware word	320.4	—

The AXI write path is **entirely free**. Every write — including the last word of each burst — completes before chip-select releases, because the write occupies 2 cycles while chip-select deassertion lags the final SCK edge by more than that. Boot latency is therefore exactly the serial shift time plus the fixed overheads.

Serial Transfer Accounting

The fetch phase is bounded entirely by serial traffic. For the benchmark image:

Table 11: Serial bits transferred during the fetch phase

Quantity	Value
Data bits (1,539 words $\times$ 32)	49,248
Header bits (2 bursts $\times$ (8 CMD $+$ 24 ADDR))	64
Total serial bits	**49,312**
At 10 system cycles per SPI bit	493,120 cycles
Measured chip-select assertion	**493,119 cycles**

Protocol headers account for 64 of 49,312 serial bits — **0.13%** of all traffic. Effectively the entire serial budget carries payload.

Scaling to Larger Images

Because boot is SPI-bound, latency scales linearly with image size at 320.4 cycles per word, on top of a fixed overhead of approximately 62,800 cycles (SRAM clear pass plus flash t_{PU}):

Table 12: Projected cold-boot latency versus image size (32 KB $+$ 32 KB SRAM, 10 MHz SCK)

Image (words)	Chunks	Cycles	Time
1,539 *(measured)*	2	555,917	5.56 ms
4,096	4	$\sim$1,375,000	$\sim$13.8 ms
8,192	8	$\sim$2,687,000	$\sim$26.9 ms
16,384 *(full SRAM)*	16	$\sim$5,312,000	$\sim$53.1 ms

Rows other than the measured 1,539-word case are projections computed from the measured per-word cost.

Note

Warm boot bypasses all of the above and reaches `S_DONE` in **6 cycles** (60 ns) — five orders of magnitude faster than cold boot, and the reason the dual-reset topology exists.

Verification

Testbench Topology

`bootloader_tb_all_testcases_32KB_sram.sv` instantiates the full boot-critical chain. Only the AXI-Lite backing memory is a behavioural stand-in; the SPI master is production RTL and the flash is the Infineon vendor model.



Figure 4: Verification environment

Testcases

Table 13: Testcase coverage

ID	Name	Checks
TC1	Cold boot	All 1,539 words match golden; burst count = 2; word count = 1,539; emits full cycle profile.
TC2	Warm boot	SPI fetch skipped; SRAM retained; reaches `S_DONE` in 6 cycles.
TC3	Determinism	Repeat cold boot must yield an identical cycle count (555,917 both runs). Catches any nondeterminism in FIFO backpressure.
TC4	Reset hold	`core_rst_ni` and `periph_rst_ni` LOW throughout boot.
TC6	APB passthrough	SPI STATUS readable through the slave port after `boot_done`.
TC8	SPI protocol monitor	Opcode `0x03`; **each burst's start address continues exactly where the previous ended**; every 32-bit word checked as its final data bit arrives; burst must not end mid-word.

Note

TC8's inter-burst address continuity check is the assertion that specifically validates the burst architecture: it proves the design genuinely relies on the flash's internal address auto-increment across a multi-word transaction, rather than re-addressing per word.

Results

Table 14: Testbench closure

Check	Result
TC1 golden image match (1,539 words)	PASS
TC1 burst count (2: 1024 + 515 words)	PASS
TC3 determinism (555,917 both runs)	PASS
TC2 warm-boot SRAM retention	PASS
TC6 APB passthrough	PASS
TC8 opcode / burst addresses / all words	PASS
Overall	**ALL PASS**

Known Coverage Gaps

- Images at or beyond the `MAX_CHUNK_WORDS` boundary (exact-multiple and 2,047-word edge cases) are reasoned about but not stimulated.
- RX FIFO clock-gating backpressure is never exercised — the drain margin is $\sim$100$\times$, so `WAIT_FIFO` is not entered. Provoking it requires an artificially slow AXI responder.
- AXI error responses (`b.resp` $\neq$ `OKAY`) and APB `pslverr` propagation.
- Quad-SPI mode and dummy-cycle read opcodes.
- Warm-boot arrival during an in-flight cold boot.

Important

The items above are known coverage gaps, not defects. Note in particular that boot latency is 88.7% serial-shift time with only 30 cycles of FSM overhead — no further restructuring of the Boot FSM can materially reduce boot time. Only SCK frequency or bits-per-clock (quad-mode) would move the dominant term.